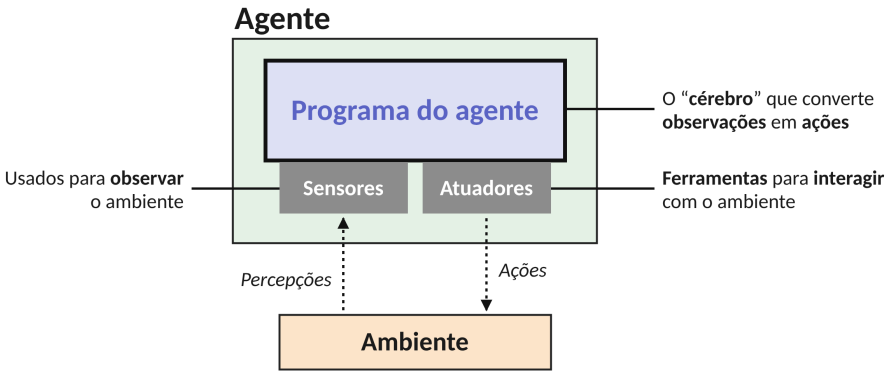
Unidade 6 — Agentes LLM

Aplicações de Modelos de Linguagem Natural

Definição de agente: agente e ambiente

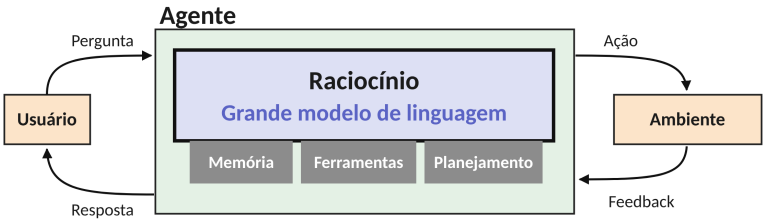
O agente recebe percepções do ambiente pelos sensores, o programa do agente decide o que fazer, e os atuadores executam ações sobre o ambiente.



Definição de agente: e os LLMs?

Vamos relacionar essa terminologia aos LLMs:

- Em um agente baseado em LLM, o **programa do agente** (o cérebro) é um **LLM de raciocínio**, capaz de pensamento complexo.
- Por meio de módulos adicionais (**memória, ferramentas e planejamento**), esse LLM consegue interagir com o ambiente.
- Os **atuadores são as ferramentas** do LLM.
- Os LLMs que interpretam mais do que texto, como imagens ou som (multimodais), podem ser vistos como os **sensores**.



O LLM como "cérebro" do agente: previsão e evolução

- Tradicionalmente, um LLM é um modelo que apenas **prevê a próxima palavra** a partir de um texto de entrada. Ele divide a entrada em **tokens** (pedaços de palavras, que permitem lidar até com palavras que nunca viu), processa esses tokens e prevê qual deve ser o próximo.
- Em seguida, acrescenta o token previsto à entrada e repete o processo. Esse ciclo, chamado de **autorregressão**, permite gerar respostas inteiras, um token por vez.

Os LLMs evoluíram bastante nos últimos anos:

- Um grande avanço veio com os **LLMs de raciocínio** (*reasoning LLMs*), como o OpenAI o1 e o DeepSeek-R1. Em vez de "pensar" de forma implícita, eles foram treinados para "pensar em voz alta": geram **tokens de raciocínio** (os "pensamentos") antes da resposta final. Assim, gastam mais computação para estruturar o problema em subproblemas, o que facilita perguntas de vários passos. Em aplicações como o ChatGPT, esses pensamentos costumam ficar ocultos ou resumidos para o usuário.

O LLM como "cérebro" do agente: por que o raciocínio importa

- O raciocínio é central para os agentes. Fazer planos extensos, escolher as ferramentas adequadas, refletir sobre erros e revisar planos são comportamentos que exigem raciocínio avançado.
- LLMs de raciocínio se saem melhor em **decisões complexas**, em **problemas de vários passos** e em **problemas novos**.
- Mas, quando se quer **respostas rápidas e baratas**, os LLMs "comuns" são preferíveis.

Ligação com a Unidade 2

Essa ideia já apareceu na Unidade 2: a técnica **Chain-of-Thought (CoT)** pedia ao modelo, no próprio prompt, que explicitasse o seu raciocínio passo a passo. Os LLMs de raciocínio foram treinados para fazer isso por conta própria, sem precisar de uma instrução específica.

Aumentando o LLM: memória

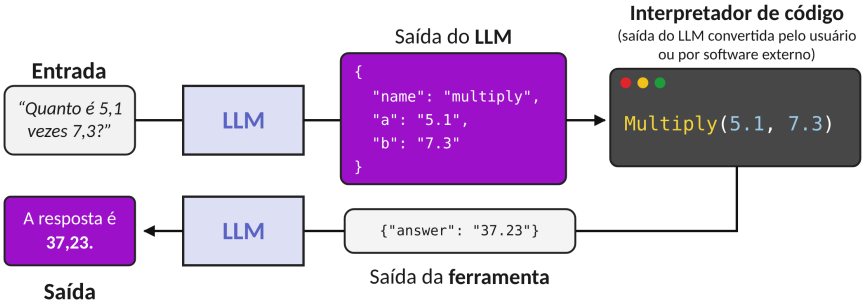
Mesmo um LLM de raciocínio é incompleto. Como entidade do tipo "texto entra, texto sai", ele não controla o ambiente, não se lembra das interações e não aprende com elas. Para resolver isso, o LLM recebe alguns "aumentos".

Memória

- Em conversas de vários turnos, aparece uma falha importante: o LLM **não guarda estado** (*stateless*), isto é, não se lembra das conversas anteriores. Sem memória, o LLM é só uma "máquina de respostas".
- Uma forma comum de simular memória é **acrescentar a conversa anterior ao prompt atual**. Na prática, os módulos de memória podem ser bem mais complexos e se parecem com a memória humana, com memória de curto e de longo prazo.
- Assim como nós, o LLM sofre com a **sobrecarga de informação**: memorizar todos os detalhes pode piorar o desempenho. É preciso equilibrar a quantidade e a qualidade da informação no prompt, o que se chama **engenharia de contexto** (detalhada na Seção 3).

Aumentando o LLM: ferramentas

- Com memória, o LLM se lembra das conversas, mas ainda não interage com o ambiente. Ferramentas externas, de uma calculadora ou um buscador até o acesso ao terminal e ao ambiente de programação, ampliam as suas capacidades.
- Porém, **o LLM não executa ferramentas sozinho**. Como função "texto entra, texto sai", ele só consegue **expressar a intenção** de agir.



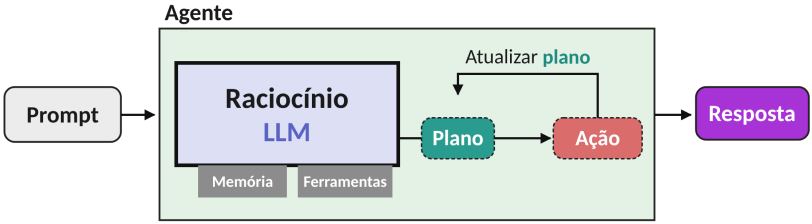
Aumentando o LLM: planejamento e reflexão

- O ingrediente final para ir de um LLM "comum" a um agente é a capacidade de **planejar e refletir**. Se o LLM tem acesso a dezenas de ferramentas (por exemplo, todas as funções da API do GitHub), como ele decide qual usar?
- O planejamento consiste em dividir uma tarefa grande em passos menores e executáveis.
- Mas criar o plano não basta: no meio do caminho, o agente pode descobrir que alguns passos não são adequados.
- Ao **refletir** sobre o próprio comportamento, o agente tenta encontrar os seus erros e corrigi-los, criando um ciclo de **planejar, agir e refletir**.

Exemplo

Um agente que pesquisa artigos descobre que o Google e o arXiv são insuficientes e acrescenta ao plano a busca no **Semantic Scholar** e no **PubMed**.

Aumentando o LLM: o que é, afinal, um agente de IA



Definição de agente

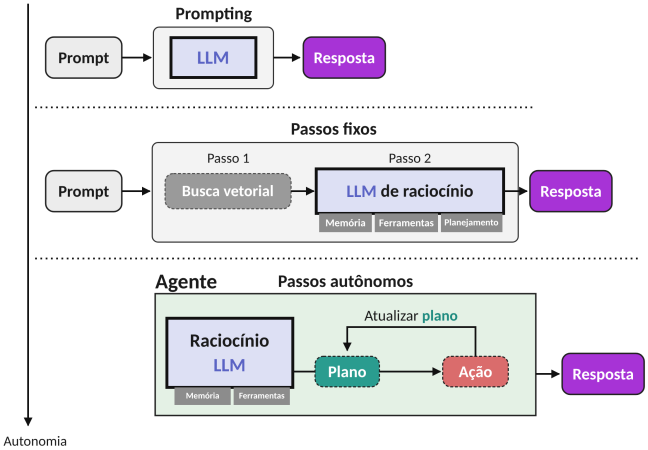
Um **LLM de raciocínio** aumentado com **memória, ferramentas, planejamento e reflexão** é o que consideramos um **agente de IA**.

Note que o ciclo "pensar, agir, observar" da Figura 4 é exatamente a ideia do **ReAct**, vista na Unidade 2.

Autonomia

- O agente tem bastante liberdade: pode escolher entre ferramentas, atualizar o plano, acrescentar passos ou parar quando chegou a uma resposta adequada. Esse comportamento é a **autonomia** concedida ao agente.
- Na prática, nem todo agente tem autonomia total. Muitas vezes são necessários **guardrails** para que o modelo não tome ações potencialmente destrutivas, como **apagar arquivos importantes**.
- A autonomia existe em um **espectro**. Ela pode ser parcial (o modelo executa um único passo, mas escolhe livremente entre ferramentas) ou total, sem nenhuma restrição.

Autonomia: três níveis



Uso responsável de agentes

Como os agentes podem agir diretamente no mundo digital ou físico, é preciso considerar as suas implicações éticas. Tudo depende do contexto: um agente de programação pode ser muito útil, mas um agente que diagnostica pacientes sem intervenção humana é, com a tecnologia atual, prejudicial. Alguns pontos a considerar:

- **Humano no circuito (*human in the loop*):** quanto mais autônomo o agente, maior a necessidade de pessoas autorizarem, verificarem e auditarem as suas decisões, normalmente checando a saída ou os passos intermediários antes de continuar.
- **Guardrails:** a autonomia total pode ser exagerada para a tarefa e até prejudicial. Um sistema com mais guardrails costuma ser mais eficaz, pois direciona o agente para os comportamentos esperados.
- **Desinformação:** agentes continuam sendo LLMs e podem gerar informações incorretas com confiança (alucinações). Sistemas em que a informação correta é crítica precisam de verificações adicionais.

Avaliação de agentes

Avaliar LLMs já é difícil, e os agentes tornam isso ainda mais difícil, porque raciocinam em vários passos, chamam ferramentas e executam sequências de ações.

- **Resultado:** a tarefa foi de fato concluída?
- **Trajetória:** os passos e as chamadas de ferramentas foram eficientes e corretos?

Duas propriedades extras merecem atenção:

- **Confiabilidade:** o agente acerta sempre, e não só uma vez, já que as suas saídas são aleatórias?
- **Segurança:** ele evita danos vindos de um usuário mal-intencionado, de dados manipulados que lê ou dos seus próprios erros?

Por isso, avaliar um agente é avaliar um **sistema inteiro**, e não apenas um modelo.